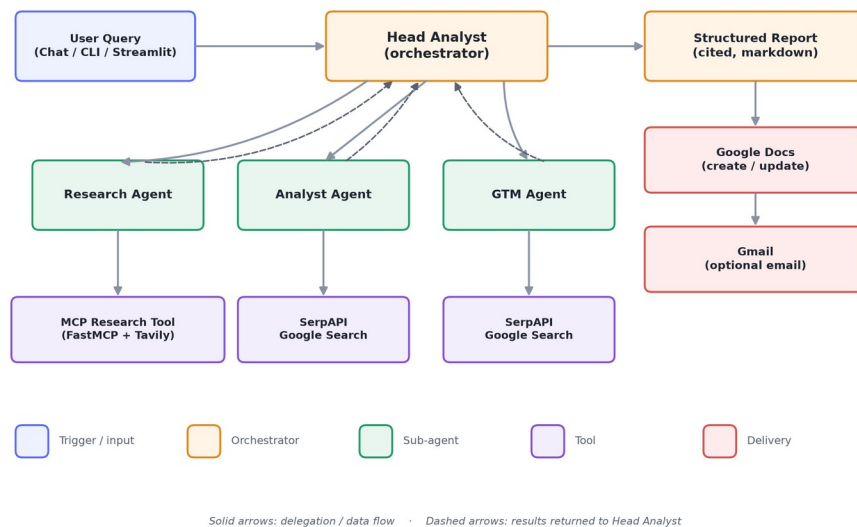


Building a Multi-Agent AI Workflow

with n8n and CrewAI (Google & LLM Integration)

Capstone Project Documentation

Market Research & Go-To-Market (GTM) Planning — Multi-Agent System



Repository: MultiAgentWorkflow_Capstone
Author: Avinash Girish

Table of Contents

1. Project Overview
2. Objectives
3. System Architecture
4. Implementation — n8n Workflow
5. Implementation — CrewAI Project
6. Design Decisions & Rationale
7. Setup & Usage Instructions
8. Testing & Validation
9. Repository Structure
10. Conclusion & Future Work

1. Project Overview

This project implements a multi-agent AI workflow that automates market research and go-to-market (GTM) planning. Given a topic (for example, “simplilearn vs edureka”), the system researches the space, builds a competitive analysis, drafts a GTM strategy, compiles everything into a single cited report, publishes that report to a Google Doc, and can optionally email the link via Gmail.

The workflow is implemented twice, to the same specification, using two different technology stacks:

- A no-code n8n workflow using n8n's AI Agent nodes, Google Docs/Gmail integrations, and a self-hosted MCP (Model Context Protocol) server.
- A CrewAI Python project with a FastMCP research backend, a command-line interface, and a Streamlit web interface.

Both implementations share the same four-agent design (one orchestrator, three specialists) and the same external integrations (Gemini/OpenAI/Azure LLMs, SerpAPI, Google Docs, Gmail), so the two versions can be compared directly.

2. Objectives

- Design a multi-agent system where a Head Analyst agent orchestrates specialist sub-agents rather than doing all the work itself.
- Ground every claim in a cited, retrievable source URL, using live web search rather than the model's own memory.
- Produce a single structured deliverable (a Google Doc) instead of a raw chat transcript.
- Implement the identical workflow twice — once no-code (n8n), once as code (CrewAI) — to compare the two approaches directly.
- Support multiple LLM back-ends (Gemini, OpenAI, Azure OpenAI) without changing the agent logic.

3. System Architecture

Both implementations follow the same logical architecture. A single entry point (a chat message, a CLI argument, or a Streamlit form) reaches the Head Analyst, which delegates to three specialist agents, each backed by its own search tool. The Head Analyst then compiles their outputs into one report and hands it to the delivery stage (Google Docs, then optionally Gmail).

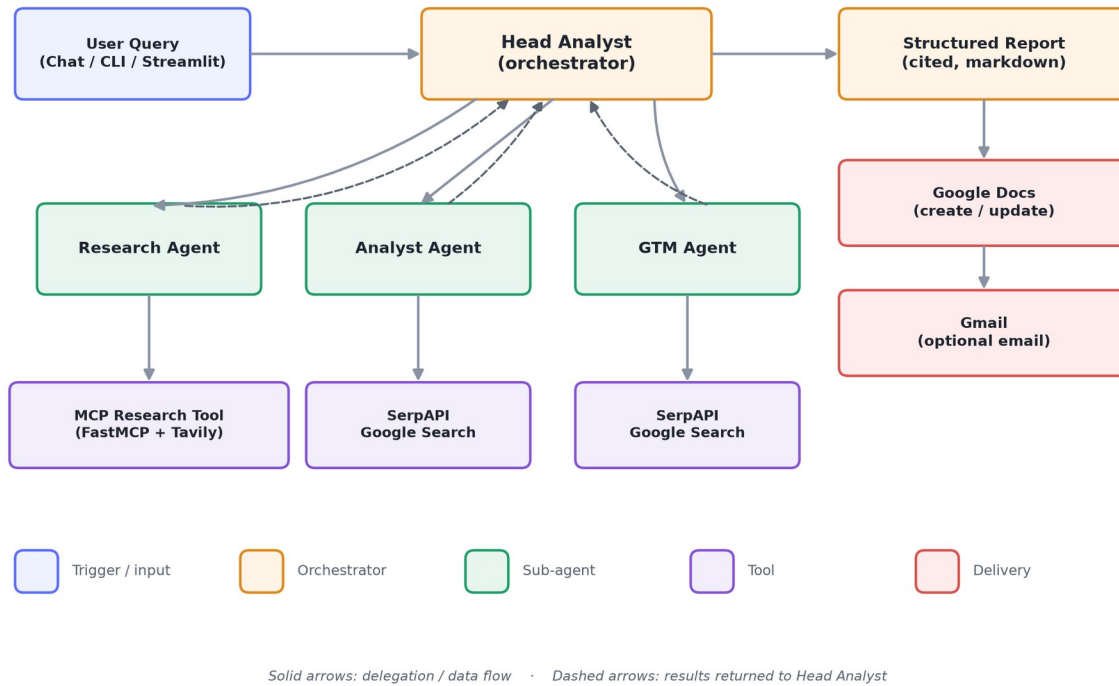


Figure 1. Multi-agent architecture shared by both the n8n and CrewAI implementations.

3.1 Agents

Agent	Responsibility	Tool(s)
Head Analyst	Orchestrates the other three agents and compiles the final cited report.	— (delegates only)
Research Agent	Finds authoritative, up-to-date sources and records their URLs.	MCP research tool (FastMCP server backed by Tavily search)
Analyst Agent	Builds a competitor comparison table and pulls out key insights.	SerpAPI Google Search
GTM Agent	Drafts the go-to-market strategy section.	SerpAPI Google Search

3.2 Data flow

1. The user submits a topic through the Chat Trigger (n8n) or the CLI/Streamlit form (CrewAI).
2. The Head Analyst delegates the topic to the Research Agent, which returns findings with source URLs.
3. The Analyst Agent receives the research and produces a competitor comparison table and insights.

4. The GTM Agent receives the research and analysis and drafts a GTM strategy section.
5. The Head Analyst compiles all three outputs, preserving every citation, into one structured report.
6. The report is published to a Google Doc (created, then updated with the full content).
7. If requested, the Google Doc link is emailed to the user via Gmail.

4. Implementation — n8n Workflow

Location in repository:

```
n8n/  
  README.md                # full setup walkthrough  
  multi_agent_analysis_workflow.json # main workflow (importable)  
  mcp_server_workflow.json  # standalone MCP server workflow
```

The main workflow's Chat Trigger feeds the Head Analyst AI Agent node. The three specialist agents are themselves AI Agent nodes, but instead of sitting in the main data flow they are connected to the Head Analyst's tool input (an n8n “ai_tool” connection). This is the same pattern n8n uses for any tool: the Head Analyst's system prompt tells it when to call “Research Agent tool”, “Analyst Agent tool”, and “GTM Agent tool”, and n8n's function-calling machinery routes the call to the corresponding sub-agent, filling its prompt automatically via `{{ $fromAI(...) }}`.

Google Doc creation and updating are wired the same way — as two more tools available to the Head Analyst — which matches the system prompt instruction to “write it to a Google Doc using doc tools.” Because the system prompt also instructs the Head Analyst to output nothing but the Doc link, that clean final answer can be piped directly into a Gmail node's message body with no further parsing.

Each agent has its own Gemini Chat Model node (swappable for OpenAI or Azure OpenAI) and, where relevant, its own SerpAPI or MCP tool node:

- Research Agent → MCP Tool node → the dedicated MCP server workflow.
- Analyst Agent → SerpAPI node (competitor / feature searches).
- GTM Agent → SerpAPI node (market / channel searches).

The MCP server workflow is a second, standalone n8n workflow: an MCP Server Trigger node exposes a connected SerpAPI node as an MCP tool, which the Research Agent's MCP Tool node calls over HTTP. This mirrors the CrewAI side's server.py, and demonstrates the same MCP integration pattern in the no-code tool.

Full step-by-step setup (Google Cloud OAuth, node-by-node configuration, testing) is documented in n8n/README.md.

5. Implementation — CrewAI Project

```
crewai_project/
pyproject.toml      # uv-managed dependencies
.env.example        # copy to .env and fill in your keys
server.py           # FastMCP research backend (Tavily search tool)
test_crewai.py      # CLI + Streamlit entrypoint
crew_app/
  config.py          # env config + LLM factory (Gemini/OpenAI/Azure)
  agents.py          # Head Analyst + 3 sub-agents
  tasks.py           # research -> analysis -> GTM -> report pipeline
  crew.py            # builds & runs the crew, then publishes the result
google_workspace.py # Google Docs create/update + Gmail send
```

5.1 Agents and tasks

crew_app/agents.py defines the four agents from Section 3.1 as CrewAI Agent objects, each with its own role, goal, backstory, and LLM. crew_app/tasks.py chains four Task objects sequentially, with each task's context set to the previous tasks so their outputs flow forward automatically: research_task → analysis_task → gtm_task → report_task (assigned to the Head Analyst).

A sequential process (crewai.Process.sequential) was chosen over CrewAI's hierarchical process for reliability: the pipeline order is fixed and known in advance (research, then analysis, then GTM, then compile), so there is no need for a manager agent to reason about which specialist to call next — that reasoning step only adds latency and a chance of misrouting without adding any real flexibility here.

5.2 MCP research backend (server.py)

server.py is a standalone FastMCP server exposing one tool, web_research(query, max_results), backed by the Tavily search API. It runs over the SSE transport on its own port and is kept running in a separate terminal, exactly like the n8n MCP server workflow. The Research Agent connects to it through crewai_tools.MCPServerAdapter, which discovers the tool automatically at startup.

5.3 Google Docs + Gmail delivery

Unlike the n8n version, Google Docs and Gmail are not exposed to the agents as callable tools. Instead, crew_app/crew.py calls crew_app/google_workspace.py directly, in plain Python, after the crew finishes. This is a deliberate reliability choice: publishing the finished report should never depend on an LLM correctly deciding to invoke a tool call. The agents focus entirely on producing good content; the orchestration code guarantees it gets delivered. crew_app/google_workspace.py uses the standard OAuth 2.0 Desktop App flow (google-auth-oauthlib), the Google Docs API's batchUpdate endpoint to insert/replace document text, and the Gmail API to send a base64-encoded MIME message.

5.4 Interfaces

test_crewai.py serves as both entrypoints:

- CLI: `uv run python test_crewai.py "simplilearn vs edureka"` — runs the crew once and prints the report and Doc link.
- Streamlit: `uv run streamlit run test_crewai.py` — a form to enter a topic, choose whether to publish to Google Docs, and optionally provide a recipient email.

The same module detects which mode it is running under and dispatches accordingly, so there is exactly one code path per interface with no duplicated logic.

6. Design Decisions & Rationale

n8n concept	CrewAI equivalent	Notes
Chat Trigger	CLI argument / Streamlit form	Same role: entry point for the user's topic.
Head Analyst tool-calling Research/Analyst/GTM agents	Sequential Task pipeline with context= chaining	n8n uses agentic tool-calls for flexibility in a no-code canvas; CrewAI uses a fixed sequential pipeline for a deterministic, testable script.
MCP Tool node -> MCP Server workflow	MCPServerAdapter -> server.py (FastMCP)	Identical integration pattern, same protocol (MCP over SSE), different runtime.
Google Doc create/update as agent tools	Deterministic Python call after crew.kickoff()	n8n showcases tool-calling; CrewAI trades that flexibility for delivery reliability.
Gmail node reading the agent's final answer	<code>google_workspace.send_email()</code> called from <code>crew.py</code>	Both only fire when an email recipient is supplied.
Gemini Chat Model node (swap for OpenAI/Azure)	<code>crew_app/config.py::get_llm()</code> (LLM_PROVIDER env var)	Same three providers supported on both sides.

7. Setup & Usage Instructions

7.1 Shared prerequisite: Google Cloud OAuth

Both implementations reuse one Google Cloud OAuth 2.0 client:

1. Create an OAuth Client ID in Google Cloud Console (Web application for n8n hosted on a server, or Desktop app for local use).
2. Enable the Gmail API and Google Docs API for the project.
3. If the OAuth consent screen is in Testing mode, add your Google account as a Test User.

Full details are in `n8n/README.md`, Step 1 (shared by both implementations).

7.2 Running the n8n workflow

1. Import `multi_agent_analysis_workflow.json` and `mcp_server_workflow.json` into your n8n instance.
2. Reattach credentials on each node (Gemini, SerpAPI, Google Docs, Gmail) — these are never portable between n8n instances.
3. Activate the MCP server workflow, then the main workflow.
4. Open the Chat Trigger's chat panel and send a topic, e.g. "simplilearn vs edureka".

7.3 Running the CrewAI project

1. `cd crewai_project && uv sync`
2. Copy `.env.example` to `.env` and fill in `LLM_PROVIDER` + its API key, `SERPAPI_API_KEY`, `TAVILY_API_KEY`.
3. Place your Google OAuth client secret as `credentials.json` in this folder.
4. Terminal 1: `uv run server.py` (starts the MCP research backend and keeps it running).
5. Terminal 2: `uv run python test_crewai.py "simplilearn vs edureka" (CLI)` or `uv run streamlit run test_crewai.py` (Streamlit).

8. Testing & Validation

The CrewAI project was built and verified against the real, currently published versions of every dependency (`crewai`, `crewai-tools`, `fastmcp`, `tavily-python`, `streamlit`, `google-api-python-client`), installed with `uv` into the project's own virtual environment. The following was verified directly, without needing live LLM/API credentials:

- `server.py` starts under FastMCP's SSE transport and accepts a connection (verified via a live SSE handshake on localhost).
- `crewai_tools.MCPDataAdapter` successfully connects to the running `server.py` and discovers the `web_research` tool by name.
- All four Agent objects, all four Task objects, and the Crew itself construct successfully with the SerpAPI and MCP tools attached, for all three LLM providers (Gemini, OpenAI, Azure OpenAI).
- Every project module (`server.py`, `test_crewai.py`, and all of `crew_app/`) compiles cleanly with no syntax or import errors.

A full end-to-end run (an actual research query producing a live Google Doc and email) requires real Gemini/SerpAPI/Tavily API keys and a completed Google OAuth consent flow, none of which are available in the development sandbox used to build this project. Section 7.3 above lists the exact steps to run one once real credentials are supplied.

The n8n workflow JSON files were hand-built against the documented n8n node schema (`chatTrigger`, `agent`, `ImChatGoogleGemini`, `mcpClientTool`, `mcpTrigger`, `toolSerpApi`, `googleDocs`, `gmail`) and validated structurally — every connection

between nodes was checked programmatically to confirm it references a node that exists in the file. A live n8n instance was not available in the sandbox to perform an interactive import test, so node versions should be reviewed after import, as noted in n8n/README.md.

9. Repository Structure

```
MultiAgentWorkflow_Capstone/  
  README.md           # top-level overview, links to both parts  
  n8n/  
    README.md  
    multi_agent_analysis_workflow.json  
    mcp_server_workflow.json  
  crewai_project/  
    README.md  
    pyproject.toml, uv.lock  
    server.py  
    test_crewai.py  
    crew_app/  
      config.py, agents.py, tasks.py, crew.py, google_workspace.py
```

10. Conclusion & Future Work

This project delivers one workflow specification implemented twice: a no-code n8n workflow for fast iteration and visual debugging, and a CrewAI Python project for programmatic control, testability, and a custom UI. Both share the same four-agent design, the same MCP-based research tool, and the same Google Docs/Gmail delivery step, making the two versions directly comparable.

Natural next steps:

- Run a full end-to-end pass with live API keys and attach the resulting Google Doc and n8n execution log as evidence.
- Add automated tests around crew_app (mocking the LLM and search calls) to catch regressions without spending API quota.
- Extend the Analyst/GTM agents with a shared caching layer so repeated topics don't re-run identical searches.